

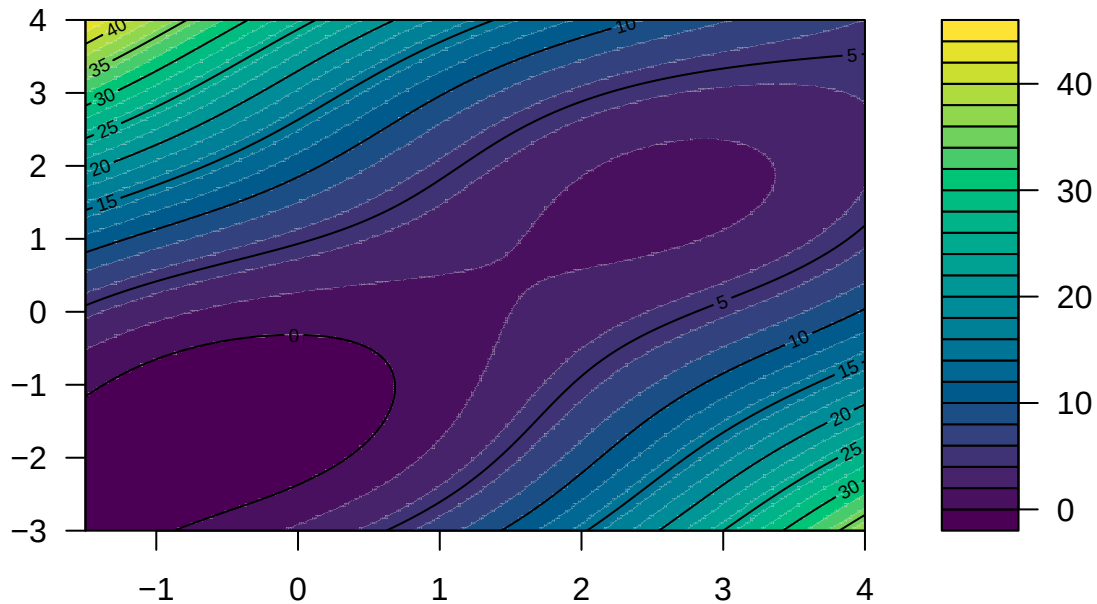
Lab 2

Aron Enström (aroen488), Sergey Volchkov (servo519)

2026-01-28

1

1.1. Function plot



1.2. Gradient and Hessian

For

$$f(x, y) = \sin(x + y) + (x - y)^2 - 1.5x + 2.5y + 1,$$

the gradient is

$$\nabla f(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix} = \begin{pmatrix} \cos(x + y) + 2(x - y) - 1.5 \\ \cos(x + y) - 2(x - y) + 2.5 \end{pmatrix}.$$

The Hessian matrix (matrix of second partial derivatives) is

$$\nabla^2 f(x, y) = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix} = \begin{pmatrix} -\sin(x+y) + 2 & -\sin(x+y) - 2 \\ -\sin(x+y) - 2 & -\sin(x+y) + 2 \end{pmatrix},$$

where $\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}$.

1.3 Newton algorithm

We are asked to search for the global minimum solution within the box

$$x \in [-1.5, 4], \quad y \in [-3, 4].$$

so the algorithm code needs to be adjusted to prevent the search path wandering outside this box.

Compared to the original (unconstrained, “pure”) Newton algorithm, we introduced three standard safeguards and one additional fallback so the method behaves well on a bounded, potentially nonconvex objective.

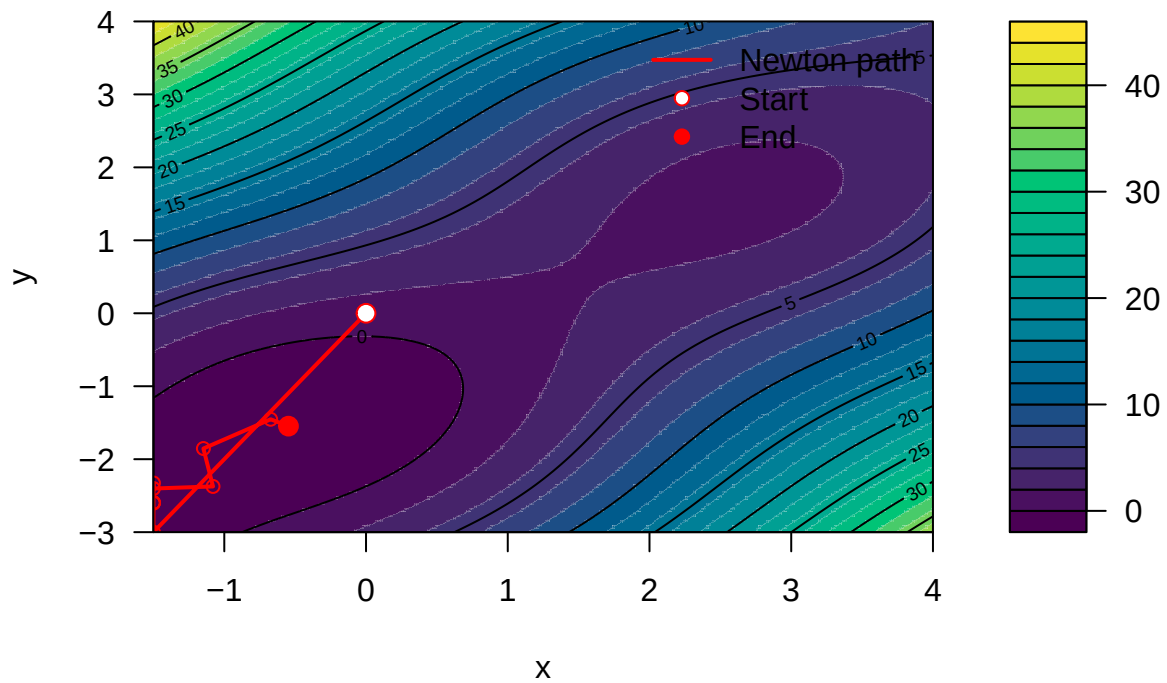
First, we enforced the box constraints by *projecting* every proposed iterate back into the feasible region: after forming a candidate update, we replace it by its componentwise clipped version so that $x \in [-1.5, 4]$ and $y \in [-3, 4]$ always hold.

Second, we stabilized the Newton step computation by *regularizing* the Hessian. Instead of solving $H(x_k, y_k) p_k = \nabla f(x_k, y_k)$ (which can fail when H is singular or ill-conditioned), we solve $(H(x_k, y_k) + \lambda I) p_k = \nabla f(x_k, y_k)$ and increase λ as needed until the linear system is solvable.

Third, we added a *backtracking line search* (step damping). Rather than always taking a full Newton step (i.e., step size $\alpha = 1$), we update via $(x_{k+1}, y_{k+1}) = P((x_k, y_k) - \alpha p_k)$ (where $P(\cdot)$ denotes the projection operator) and reduce α (e.g., $\alpha \leftarrow \beta \alpha$ with $0 < \beta < 1$) until an Armijo-type sufficient decrease condition is satisfied, which prevents overshooting and improves robustness on nonconvex landscapes.

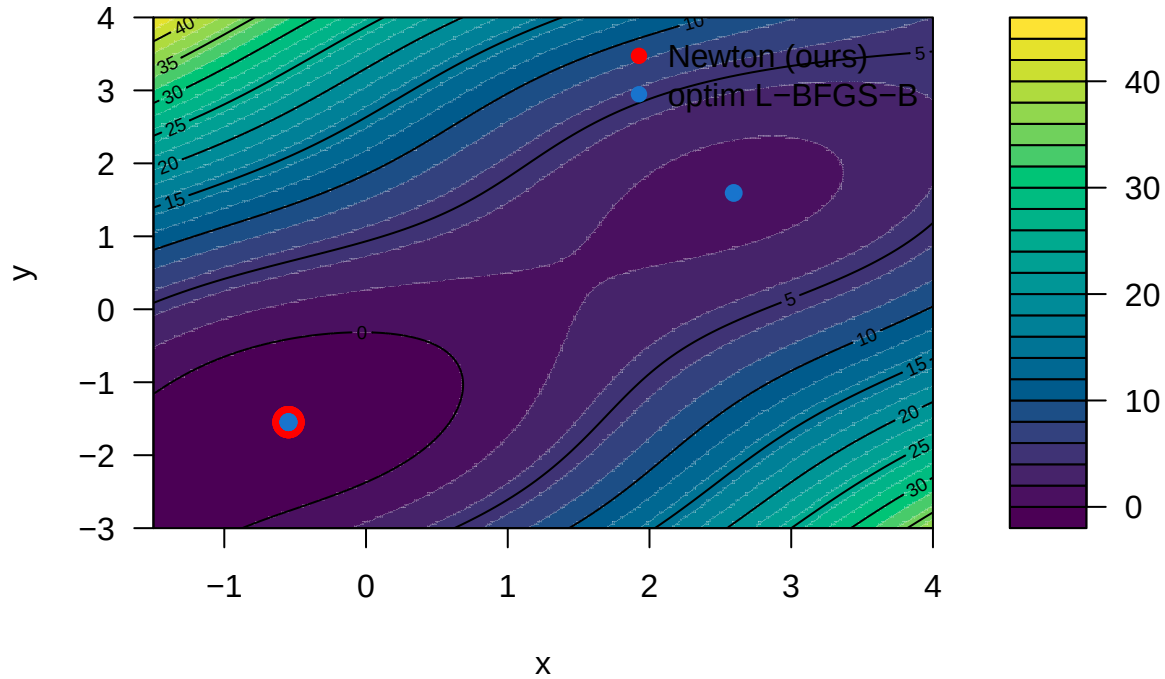
Finally, to avoid stalling at the boundary (where projection can collapse the effective Newton step and force α to become extremely small), we included a fallback to a *projected-gradient* step when the Newton step fails to produce progress. Concretely, we switch to $(x_{k+1}, y_{k+1}) = P((x_k, y_k) - \alpha \nabla f(x_k, y_k))$, which provides a feasible descent direction and helps the algorithm move along edges instead of getting trapped at corners.

Testing convergence from starting point at (0,0)



Comparing with the results produced by L-BFGS-B algorithm (BGFS with constraints) on a small sample of points.

Comparing the results with L-BFGS-B algorithm on a sample of points

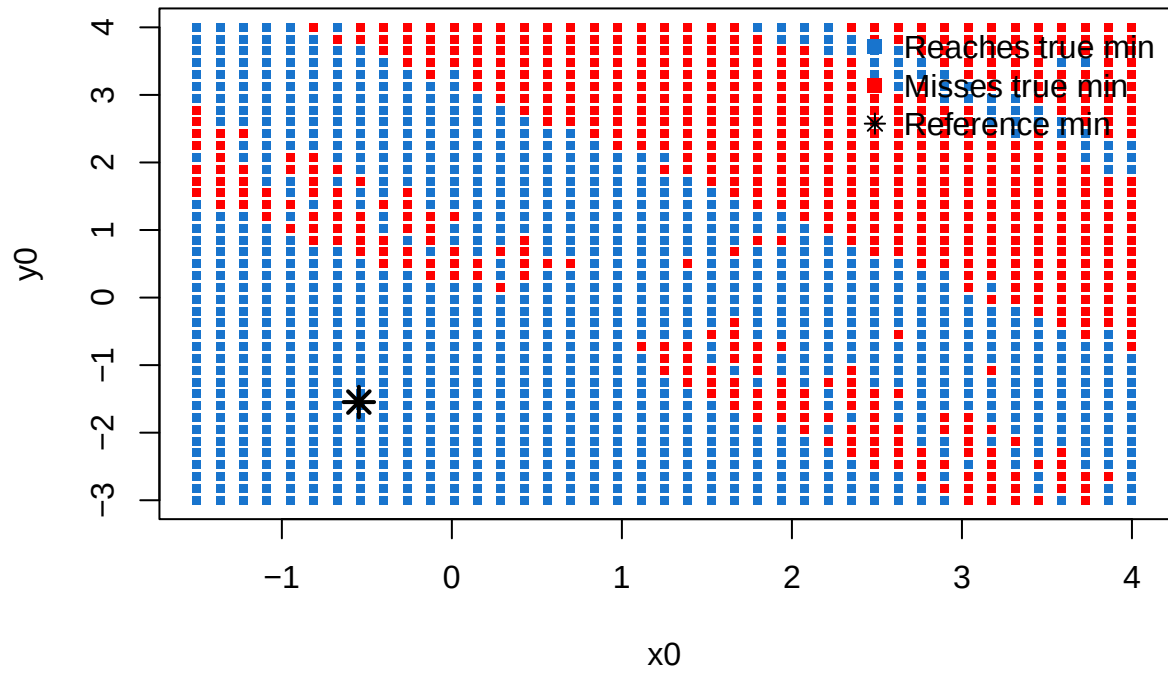


We can see from the above diagram that our algorithm converges in the same spot from all the randomly selected starting points, the endpoint seems to be where we expect the global minimum to lie on our heatmap plot and it coincides with the one produced by the L-BFGS-B algorithm which gives us confidence that it is working correctly.

1.4 Testing with different starting points

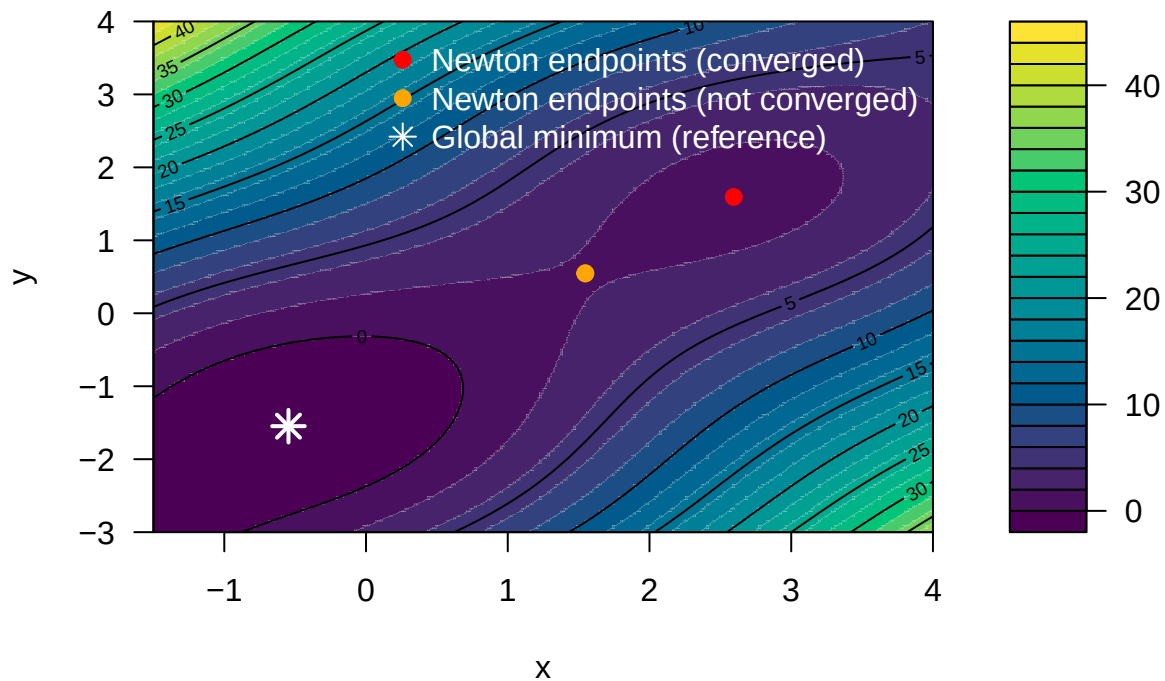
To identify the points from which the algorithms does not converge to the true minimum, we need to use a denser grid:

Starts: Newton misses (red) vs reaches (blue) the true minimum



Using the above output to see which points the algorithm converges to when it does not converge to the true minimum:

Attractors other than the global minimum



1.5 Understanding the nature of the incorrect endpoints

```
##
##      Global minimum      Local min Saddle / indefinite
##              1              1              2
##
##      end_x    end_y    end_f converged      class
## 1  -0.5471976 -1.547198 -1.913223      1      Global minimum
## x   2.5943950  1.594395  1.228370      1              Local min
## x77 1.5471670  0.547160  1.913223      0 Saddle / indefinite
## x91 1.5471450  0.547145  1.913223      0 Saddle / indefinite
```

2

Appendix

```
#1.1
f <- function(x, y) sin(x + y) + (x - y)^2 - 1.5*x + 2.5*y + 1

x <- seq(-1.5, 4, length.out = 300)
y <- seq(-3, 4, length.out = 300)

z <- outer(x, y, Vectorize(f))

filled.contour(x, y, z,
  nlevels = 30,
```

```

color.palette = hcl.colors,
plot.axes = {
  axis(1); axis(2)
  contour(x, y, z, add = TRUE, drawlabels = TRUE)
}
)
#1.2
## Gradient
grad_f <- function(x, y) {
  c(
    dx = cos(x + y) + 2*(x - y) - 1.5,
    dy = cos(x + y) - 2*(x - y) + 2.5
  )
}

## Hessian
hess_f <- function(x, y) {
  s <- -sin(x + y)
  matrix(
    c(s + 2, s - 2,
      s - 2, s + 2),
    nrow = 2, byrow = TRUE,
    dimnames = list(c("x", "y"), c("x", "y"))
  )
}

## wrappers that take par = c(x, y) for optimization
fn_par <- function(par) f(par[1], par[2])
gr_par <- function(par) grad_f(par[1], par[2])
hess_par <- function(par) hess_f(par[1], par[2])

## test
# par0 <- c(0, 0)
# fn_par(par0)
# gr_par(par0)
# hess_par(par0)
#1.3

newton2d_box <- function(fn, gr, hess, x0, y0,
  lower, upper,
  tol = 1e-8, maxit = 200,
  lambda0 = 1e-6, lambda_mult = 10,
  alpha0 = 1, beta = 0.5, c1 = 1e-4,
  pg_alpha0 = 1, pg_beta = 0.5,
  eps_active = 1e-10,
  ..., verbose = FALSE, keep_path = TRUE) {

project <- function(par) pmin(pmax(par, lower), upper)

## projected-gradient infinity norm (KKT-style for box constraints)
projgrad_inf <- function(x, g, eps = 1e-12) {
  pg <- g
  for (i in 1:2) {

```

```

    if (x[i] <= lower[i] + eps) pg[i] <- min(g[i], 0)
    if (x[i] >= upper[i] - eps) pg[i] <- max(g[i], 0)
  }
  max(abs(pg))
}

## "gradient mapping" direction:  $d = P(x - a g) - x$  (feasible descent step)
gradmap_dir <- function(x, g, a) project(x - a * g) - x

x <- project(c(x0, y0))
f0 <- fn(x[1], x[2], ...)

path <- NULL
if (keep_path) {
  path <- data.frame(iter = 0L, x = x[1], y = x[2], f = f0,
    step = NA_real_, method = "init",
    projgrad = projgrad_inf(x, gr(x[1], x[2], ...)))
}

for (k in 1:maxit) {
  g <- gr(x[1], x[2], ...)
  pg_norm <- projgrad_inf(x, g)

  if (pg_norm < tol) {
    return(list(par = x, value = f0, iter = k - 1, converged = TRUE,
      projgrad = pg_norm, path = path))
  }

  ## ---- Attempt Newton step (with regularization + projected Armijo) ----
  H <- hess(x[1], x[2], ...)
  lambda <- lambda0
  p <- NULL
  repeat {
    p_try <- tryCatch(solve(H + lambda * diag(2), g), error = function(e) NULL)
    if (!is.null(p_try)) { p <- p_try; break }
    lambda <- lambda * lambda_mult
    if (lambda > 1e12) { p <- NULL; break }
  }

  newton_ok <- FALSE
  a <- alpha0
  if (!is.null(p)) {
    repeat {
      x_new <- project(x - a * p)
      f_new <- fn(x_new[1], x_new[2], ...)
      armijo_rhs <- f0 - c1 * a * sum(g * p) # sufficient decrease
      if (is.finite(f_new) && f_new <= armijo_rhs && any(abs(x_new - x) > 0)) {
        newton_ok <- TRUE
        break
      }
    }
    a <- a * beta
    if (a < 1e-12) break
  }
}

```



```

}

if (newton_ok) {
  x <- x_new; f0 <- f_new
  if (keep_path) path <- rbind(path, data.frame(iter = k, x = x[1], y = x[2], f = f0,
                                                step = a, method = "Newton",
                                                projgrad = pg_norm))
  if (verbose) cat(sprintf("iter=%d Newton  x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e\n",
                           k, x[1], x[2], f0, a, pg_norm))
  next
}

## ---- Fallback: projected-gradient step (guarantees feasible movement) ----
a_pg <- pg_alpha0
repeat {
  d <- gradmap_dir(x, g, a_pg)
  x_new <- x + d
  f_new <- fn(x_new[1], x_new[2], ...)
  if (is.finite(f_new) && f_new <= f0) break # simple monotone decrease
  a_pg <- a_pg * pg_beta
  if (a_pg < 1e-12) break
}

## If even PG can't decrease, stop (we're at a constrained stationary-ish point)
if (sqrt(sum((x_new - x)^2)) < tol) {
  return(list(par = x, value = f0, iter = k, converged = FALSE,
             projgrad = pg_norm, path = path,
             message = "No progress (Newton+PG failed to decrease)."))
}

x <- x_new; f0 <- f_new
if (keep_path) path <- rbind(path, data.frame(iter = k, x = x[1], y = x[2], f = f0,
                                              step = a_pg, method = "PG",
                                              projgrad = pg_norm))
if (verbose) cat(sprintf("iter=%d PG      x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e\n",
                         k, x[1], x[2], f0, a_pg, pg_norm))
}

list(par = x, value = f0, iter = maxit, converged = FALSE,
     projgrad = projgrad_inf(x, gr(x[1], x[2], ...)),
     path = path)
}

# newton2d_box <- function(fn, gr, hess, x0, y0,
#                           lower, upper,
#                           tol = 1e-8, maxit = 100,
#                           lambda0 = 1e-6, lambda_mult = 10,
#                           alpha0 = 1, beta = 0.5, c1 = 1e-4,
#                           eps_active = 1e-10,
#                           ..., verbose = FALSE, keep_path = TRUE) {
#
#   project <- function(par) pmin(pmax(par, lower), upper)
#

```

```

# projgrad_inf <- function(x, g, eps = 1e-12) {
#   pg <- g
#   for (i in 1:2) {
#     if (x[i] <= lower[i] + eps) pg[i] <- min(g[i], 0)
#     if (x[i] >= upper[i] - eps) pg[i] <- max(g[i], 0)
#   }
#   max(abs(pg))
# }
#
# ## Active-set: free if not too close to either bound
# free_set <- function(x) {
#   c(
#     (x[1] > lower[1] + eps_active) && (x[1] < upper[1] - eps_active),
#     (x[2] > lower[2] + eps_active) && (x[2] < upper[2] - eps_active)
#   )
# }
#
# x <- project(c(x0, y0))
# f0 <- fn(x[1], x[2], ...)
#
# path <- NULL
# if (keep_path) {
#   path <- data.frame(iter = 0L, x = x[1], y = x[2], f = f0,
#     step = NA_real_, projgrad = NA_real_, lambda = NA_real_,
#     free_x = NA, free_y = NA)
# }
#
# for (k in 1:maxit) {
#   g <- gr(x[1], x[2], ...)
#   H <- hess(x[1], x[2], ...)
#   pg_norm <- projgrad_inf(x, g)
#
#   free <- free_set(x)
#   if (keep_path) {
#     path <- rbind(path, data.frame(iter = k, x = x[1], y = x[2], f = f0,
#       step = NA_real_, projgrad = pg_norm, lambda = NA_real_,
#       free_x = free[1], free_y = free[2]))
#   }
#
#   if (pg_norm < tol) {
#     return(list(par = x, value = f0, iter = k - 1, converged = TRUE,
#       projgrad = pg_norm, path = path))
#   }
#
#   ## If nothing is free, we're at a corner/edge; stop (or switch to PG)
#   if (!any(free)) {
#     return(list(par = x, value = f0, iter = k - 1, converged = FALSE,
#       projgrad = pg_norm, path = path,
#       message = "Stuck at corner: no free variables."))
#   }
#
#   ## Reduced Newton direction p: only update free components
#   p_full <- c(0, 0)

```

```

#   idx <- which(free)
#
#   lambda <- lambda0
#   repeat {
#     Hred <- H[idx, idx, drop = FALSE]
#     gred <- g[idx]
#     p_try <- tryCatch(solve(Hred + lambda * diag(length(idx)), gred),
#                       error = function(e) NULL)
#     if (!is.null(p_try)) { p_full[idx] <- p_try; break }
#     lambda <- lambda * lambda_mult
#     if (lambda > 1e12) stop("Could not compute reduced Newton step.")
#   }
#
#   ## Line search on reduced step (projection just guards numerics)
#   a <- alpha0
#   repeat {
#     x_new <- project(x - a * p_full)
#     f_new <- fn(x_new[1], x_new[2], ...)
#
#     armijo_rhs <- f0 - c1 * a * sum(g[idx] * p_full[idx]) # uses free components
#     if (is.finite(f_new) && f_new <= armijo_rhs) break
#
#     a <- a * beta
#     if (a < 1e-12) break
#   }
#
#   if (verbose) {
#     cat(sprintf("iter=%d  x=%.8f y=%.8f  f=%.10f  step=%.1e  ||pg||_inf=%.3e  lambda=%.1e  free=(%d",
#                 k, x_new[1], x_new[2], f_new, a, pg_norm, lambda, free[1], free[2]))
#   }
#
#   if (keep_path) {
#     path[nrow(path), "step"] <- a
#     path[nrow(path), "lambda"] <- lambda
#   }
#
#   ## Secondary stop: tiny move AND small projected grad
#   if (sqrt(sum((x_new - x)^2)) < tol && pg_norm < tol) {
#     return(list(par = x_new, value = f_new, iter = k, converged = TRUE,
#                 projgrad = pg_norm, path = path))
#   }
#
#   ## If the step collapsed, bail out instead of looping forever
#   if (a < 1e-12) {
#     return(list(par = x, value = f0, iter = k, converged = FALSE,
#                 projgrad = pg_norm, path = path,
#                 message = "Line search failed (step underflow)."))
#   }
#
#   x <- x_new
#   f0 <- f_new
# }
#

```

```

# list(par = x, value = f0, iter = maxit, converged = FALSE,
#       projgrad = projgrad_inf(x, gr(x[1], x[2], ...)),
#       path = path)
# }

# newton2d_box <- function(fn, gr, hess, x0, y0,
#                          lower, upper,
#                          tol = 1e-8, maxit = 100,
#                          lambda0 = 1e-6, lambda_mult = 10,
#                          alpha0 = 1, beta = 0.5, c1 = 1e-4,
#                          ..., verbose = FALSE, keep_path = TRUE) {
#
#   project <- function(par) pmin(pmax(par, lower), upper)
#
#   ## infinity-norm of projected gradient (KKT-style for box constraints)
#   projgrad_inf <- function(x, g, eps = 1e-12) {
#     pg <- g
#     for (i in 1:2) {
#       if (x[i] <= lower[i] + eps) pg[i] <- min(g[i], 0)
#       if (x[i] >= upper[i] - eps) pg[i] <- max(g[i], 0)
#     }
#     max(abs(pg))
#   }
#
#   x <- project(c(x0, y0))
#   f0 <- fn(x[1], x[2], ...)
#
#   ## store path (x, y, f, step, projgrad, lambda)
#   path <- NULL
#   if (keep_path) {
#     path <- data.frame(iter = 0L, x = x[1], y = x[2], f = f0,
#                        step = NA_real_, projgrad = NA_real_, lambda = NA_real_)
#   }
#
#   for (k in 1:maxit) {
#     g <- gr(x[1], x[2], ...)
#     pg_norm <- projgrad_inf(x, g)
#
#     if (keep_path) path <- rbind(path, data.frame(
#       iter = k, x = x[1], y = x[2], f = f0,
#       step = NA_real_, projgrad = pg_norm, lambda = NA_real_
#     ))
#
#     if (pg_norm < tol) {
#       return(list(par = x, value = f0, iter = k - 1, converged = TRUE,
#                  projgrad = pg_norm, path = path))
#     }
#
#     H <- hess(x[1], x[2], ...)
#     I2 <- diag(2)
#
#     ## Regularize Hessian until solvable
#     lambda <- lambda0

```

```

#   p <- NULL
#   repeat {
#     p_try <- tryCatch(solve(H + lambda * I2, g), error = function(e) NULL)
#     if (!is.null(p_try)) { p <- p_try; break }
#     lambda <- lambda * lambda_mult
#     if (lambda > 1e12) stop("Could not compute Newton step (even after regularization).")
#   }
#
#   ## Backtracking line search with Armijo sufficient decrease
#   a <- alpha0
#   repeat {
#     x_new <- project(x - a * p)
#     f_new <- fn(x_new[1], x_new[2], ...)
#
#     armijo_rhs <- f0 - c1 * a * sum(g * p)
#
#     if (is.finite(f_new) && f_new <= armijo_rhs) break
#     a <- a * beta
#     if (a < 1e-12) break
#   }
#
#   if (verbose) {
#     cat(sprintf(
#       "iter=%d x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e lambda=%.1e\n",
#       k, x_new[1], x_new[2], f_new, a, pg_norm, lambda
#     ))
#   }
#
#   if (keep_path) {
#     path[nrow(path), "step"] <- a
#     path[nrow(path), "lambda"] <- lambda
#     path[nrow(path), "projgrad"] <- pg_norm
#   }
#
#   ## Only use step-size convergence as a secondary criterion
#   if (sqrt(sum((x_new - x)^2)) < tol && pg_norm < tol) {
#     return(list(par = x_new, value = f_new, iter = k, converged = TRUE,
#               projgrad = pg_norm, path = path))
#   }
#
#   x <- x_new
#   f0 <- f_new
# }
#
# list(par = x, value = f0, iter = maxit, converged = FALSE,
#      projgrad = projgrad_inf(x, gr(x[1], x[2], ...)),
#      path = path)
# }

# newton2d_box <- function(fn, gr, hess, x0, y0,
#                          lower, upper,
#                          tol = 1e-8, maxit = 100,
#                          lambda0 = 1e-6, lambda_mult = 10,

```

```

#             alpha0 = 1, beta = 0.5, c1 = 1e-4,
#             ..., verbose = FALSE) {
#
#   project <- function(par) pmin(pmax(par, lower), upper)
#
#   ## infinity-norm of projected gradient (KKT-style for box constraints)
#   projgrad_inf <- function(x, g, eps = 1e-12) {
#     pg <- g
#     for (i in 1:2) {
#       if (x[i] <= lower[i] + eps) pg[i] <- min(g[i], 0) # should be >=0 at lower bound
#       if (x[i] >= upper[i] - eps) pg[i] <- max(g[i], 0) # should be <=0 at upper bound
#     }
#     max(abs(pg))
#   }
#
#   x <- project(c(x0, y0))
#   f0 <- fn(x[1], x[2], ...)
#
#   for (k in 1:maxit) {
#     g <- gr(x[1], x[2], ...)
#     pg_norm <- projgrad_inf(x, g)
#
#     if (pg_norm < tol) {
#       return(list(par = x, value = f0, iter = k - 1, converged = TRUE,
#                 projgrad = pg_norm))
#     }
#
#     H <- hess(x[1], x[2], ...)
#     I2 <- diag(2)
#
#     ## Regularize Hessian until solvable
#     lambda <- lambda0
#     p <- NULL
#     repeat {
#       p_try <- tryCatch(solve(H + lambda * I2, g), error = function(e) NULL)
#       if (!is.null(p_try)) { p <- p_try; break }
#       lambda <- lambda * lambda_mult
#       if (lambda > 1e12) stop("Could not compute Newton step (even after regularization).")
#     }
#
#     ## Backtracking line search with Armijo sufficient decrease
#     a <- alpha0
#     repeat {
#       x_new <- project(x - a * p)
#       f_new <- fn(x_new[1], x_new[2], ...)
#
#       ## directional derivative along -p is approx -g^T p
#       armijo_rhs <- f0 - c1 * a * sum(g * p)
#
#       if (is.finite(f_new) && f_new <= armijo_rhs) break
#       a <- a * beta
#       if (a < 1e-12) break
#     }
#
#   }

```

```

#
#   if (verbose) {
#       cat(sprintf(
#           "iter=%d x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e lambda=%.1e\n",
#           k, x_new[1], x_new[2], f_new, a, pg_norm, lambda
#       ))
#   }
#
#   ## Only use step-size convergence as a secondary criterion
#   if (sqrt(sum((x_new - x)^2)) < tol && pg_norm < tol) {
#       return(list(par = x_new, value = f_new, iter = k, converged = TRUE,
#                   projgrad = pg_norm))
#   }
#
#   x <- x_new
#   f0 <- f_new
# }
#
# list(par = x, value = f0, iter = maxit, converged = FALSE,
#       projgrad = projgrad_inf(x, gr(x[1], x[2], ...)))
# }

# newton2d_box <- function(fn, gr, hess, x0, y0,
#                           lower, upper,
#                           tol = 1e-8, maxit = 100,
#                           lambda0 = 1e-6, lambda_mult = 10,
#                           alpha0 = 1, beta = 0.5,
#                           ..., verbose = FALSE) {
#
#   #To keep the results within the prescribed x,y limits
#   project <- function(par) pmin(pmax(par, lower), upper)
#
#   x <- project(c(x0, y0))
#   f0 <- fn(x[1], x[2], ...)
#
#   for (k in 1:maxit) {
#       g <- gr(x[1], x[2], ...)
#       H <- hess(x[1], x[2], ...)
#       I2 <- diag(2)
#
#       ## regularize until solvable
#       lambda <- lambda0
#       p <- NULL
#       repeat {
#           p_try <- tryCatch(solve(H + lambda * I2, g), error = function(e) NULL)
#           if (!is.null(p_try)) { p <- p_try; break }
#           lambda <- lambda * lambda_mult
#           if (lambda > 1e12) stop("Could not compute Newton step.")
#       }
#
#       ## backtracking line search with projection
#       a <- alpha0

```

```

#   repeat {
#       x_new <- project(x - a * p)
#       f_new <- fn(x_new[1], x_new[2], ...)
#       if (is.finite(f_new) && f_new <= f0) break
#       a <- a * beta
#       if (a < 1e-12) break
#   }
#
#   if (verbose) {
#       cat(sprintf("iter=%d  x=%.8f y=%.8f  f=%.10f  step=%.1e\n",
#                   k, x_new[1], x_new[2], f_new, a))
#   }
#
#   if (sqrt(sum((x_new - x)^2)) < tol) {
#       return(list(par = x_new, value = f_new, iter = k, converged = TRUE))
#   }
#
#   x <- x_new
#   f0 <- f_new
# }
#
# list(par = x, value = f0, iter = maxit, converged = FALSE)
# }

# ##  $x_{k+1} = x_k - H(x_k)^{-1} * g(x_k)$ 
# newton2d_reg <- function(fn, gr, hess, x0, y0,
#                           tol = 1e-8, maxit = 100,
#                           lambda0 = 1e-6, lambda_mult = 10,
#                           ..., verbose = FALSE) {
#
#   x <- c(x0, y0)
#
#   for (k in 1:maxit) {
#       g <- gr(x[1], x[2], ...)
#       H <- hess(x[1], x[2], ...)
#       I2 <- diag(2)
#
#       lambda <- lambda0
#       p <- NULL
#
#       ## Keep increasing lambda until the linear system is solvable
#       repeat {
#           Hp <- H + lambda * I2
#           p_try <- tryCatch(solve(Hp, g), error = function(e) NULL) # catch singularity
#           if (!is.null(p_try)) { p <- p_try; break }
#           lambda <- lambda * lambda_mult
#           if (lambda > 1e12) stop("Hessian remained singular/ill-conditioned even after regularization.")
#       }
#
#       x_new <- x - p
#
#       if (verbose) {
#           cat(sprintf("iter=%d  x=%.10f y=%.10f  f=%.10f  ||grad||=%.3e  lambda=%.1e\n",

```



```

#           k, x_new[1], x_new[2], fn(x_new[1], x_new[2], ...),
#           sqrt(sum(g^2)), lambda))
#     }
#
#     if (sqrt(sum((x_new - x)^2)) < tol) {
#       return(list(par = x_new, value = fn(x_new[1], x_new[2], ...),
#         iter = k, converged = TRUE))
#     }
#     x <- x_new
#   }
#
#   list(par = x, value = fn(x[1], x[2], ...), iter = maxit, converged = FALSE)
# }
#
lower <- c(-1.5, -3)
upper <- c( 4.0,  4)
# res <- newton2d_box(fn = f, gr = grad_f, hess = hess_f,
#                   x0 = 0, y0 = 0, lower=lower, upper=upper, verbose = TRUE)
# res$par
# res$value
#
#
res00 <- newton2d_box(fn = f, gr = grad_f, hess = hess_f,
                    x0 = 0, y0 = 0,
                    lower = lower, upper = upper,
                    verbose = FALSE, keep_path = TRUE)

# head(res00$path)

filled.contour(x, y, z,
  nlevels = 30,
  color.palette = hcl.colors,
  plot.title = title(main = "Testing convergence from starting point at (0,0)",
    xlab = "x", ylab = "y"),
  plot.axes = {
    axis(1); axis(2)
    contour(x, y, z, add = TRUE, drawlabels = TRUE)

    ## Newton path: line + points (start = open circle, end = filled circle)
    lines(res00$path$x, res00$path$y, col = "red", lwd = 2)
    points(res00$path$x, res00$path$y, col = "red", pch = 1, cex = 0.9)
    points(res00$path$x[1], res00$path$y[1], col = "red", pch = 21, bg = "white", cex = 1.3)
    points(tail(res00$path$x, 1), tail(res00$path$y, 1), col = "red", pch = 19, cex = 1.3)

    legend("topright",
      legend = c("Newton path", "Start", "End"),
      col = c("red", "red", "red"),
      lwd = c(2, NA, NA),
      pch = c(NA, 21, 19),
      pt.bg = c(NA, "white", NA),
      bty = "n")
  }
}

```

```

)
lower <- c(-1.5, -3)
upper <- c( 4.0,  4)

one_run <- function(x0, y0) {
  nres <- newton2d_box(f, grad_f, hess_f, x0, y0, lower, upper)

  ores <- optim(
    par = c(x0, y0),
    fn = fn_par, gr = gr_par,
    method = "L-BFGS-B",
    lower = lower, upper = upper
  ) # box constraints supported by L-BFGS-B

  data.frame(
    x0 = x0, y0 = y0,
    newton_iter = nres$iter,
    newton_x = nres$par[1], newton_y = nres$par[2],
    newton_value = nres$value,
    optim_iter = ores$counts[["function"]],
    optim_x = ores$par[1], optim_y = ores$par[2],
    optim_value = ores$value
  )
}

starts <- rbind(
  c(-1.0, -2.0),
  c( 3.5,  3.5),
  c( 0.0,  0.0),
  c( 4.0, -3.0),
  c(-1.5,  4.0)
)
colnames(starts) <- c("x0", "y0")

res_table <- do.call(
  rbind,
  lapply(seq_len(nrow(starts)), function(i) one_run(starts[i, 1], starts[i, 2]))
)

res_table_fmt <- res_table
num_cols <- names(res_table_fmt)[sapply(res_table_fmt, is.numeric)]

res_table_fmt[num_cols] <- lapply(res_table_fmt[num_cols], \(v) formatC(v, format = "f", digits = 6))
# print(res_table_fmt, row.names = FALSE)

## distinct solutions reached (rounded to group near-equal points)
# unique(round(res_table[, c("newton_x", "newton_y")], 6))
# unique(round(res_table[, c("optim_x", "optim_y")], 6))

filled.contour(x, y, z,
  nlevels = 30,

```

```

color.palette = hcl.colors,
plot.title = title(main = "Comparing the results with L-BFGS-B algorithm\non a sample of points",
                    xlab = "x", ylab = "y"),
plot.axes = {
  axis(1); axis(2)
  contour(x, y, z, add = TRUE, drawlabels = TRUE)

  ## Overlay solutions
  points(res_table$newton_x, res_table$newton_y,
         pch = 19, col = "red", cex = 2.1)
  points(res_table$optim_x, res_table$optim_y,
         pch = 19, col = "dodgerblue3", cex = 1.1)

  legend("topright",
        legend = c("Newton (ours)", "optim L-BFGS-B"),
        col = c("red", "dodgerblue3"),
        pch = 19, bty = "n")
}
)
#1.4
lower <- c(-1.5, -3)
upper <- c( 4.0,  4)

## multi-start L-BFGS-B to approximate the global minimum in the box
x_st <- seq(lower[1], upper[1], length.out = 15)
y_st <- seq(lower[2], upper[2], length.out = 15)
starts_ref <- expand.grid(x0 = x_st, y0 = y_st) # all combinations

run_lbfgsb <- function(x0, y0) {
  o <- optim(par = c(x0, y0),
            fn = fn_par, gr = gr_par,
            method = "L-BFGS-B",
            lower = lower, upper = upper) # bound constraints
  list(par = o$par, value = o$value, counts = o$counts)
}

ref_list <- lapply(seq_len(nrow(starts_ref)), function(i) run_lbfgsb(starts_ref$x0[i], starts_ref$y0[i]))
ref_vals <- sapply(ref_list, `[`, "value")
best_i <- which.min(ref_vals)

x_star_ref <- ref_list[[best_i]]$par
f_star_ref <- ref_list[[best_i]]$value
# x_star_ref; f_star_ref

tol_val <- 1e-6

## grid (or increase resolution / use random points)
xg <- seq(lower[1], upper[1], length.out = 41)
yg <- seq(lower[2], upper[2], length.out = 41)
starts <- expand.grid(x0 = xg, y0 = yg)

one_run_newton <- function(x0, y0) {
  nres <- newton2d_box(f, grad_f, hess_f, x0, y0, lower, upper)

```

```

data.frame(
  x0 = x0, y0 = y0,
  converged = isTRUE(nres$converged),
  iters = nres$iter,
  x = nres$par[1], y = nres$par[2],
  f = nres$value,
  miss_true_min = (!isTRUE(nres$converged)) || (nres$value > f_star_ref + tol_val)
)
}

scan_res <- do.call(rbind, lapply(seq_len(nrow(starts)), function(i) one_run_newton(starts$x0[i], start.

## starting points where Newton does NOT reach the (estimated) global min
bad_starts <- subset(scan_res, miss_true_min)

# nrow(bad_starts)
# head(bad_starts)

cols <- ifelse(scan_res$miss_true_min, "red", "dodgerblue3")
plot(scan_res$x0, scan_res$y0, col = cols, pch = 15, cex = 0.6,
     xlab = "x0", ylab = "y0",
     main = "Starts: Newton misses (red) vs reaches (blue) the true minimum")
points(x_star_ref[1], x_star_ref[2], pch = 8, cex = 1.5, lwd = 2)
legend("topright", legend = c("Reaches true min", "Misses true min", "Reference min"),
     col = c("dodgerblue3", "red", "black"), pch = c(15, 15, 8), bty = "n")

endpts <- do.call(rbind, lapply(seq_len(nrow(bad_starts)), function(i) {
  x0 <- bad_starts$x0[i]
  y0 <- bad_starts$y0[i]

  r <- newton2d_box(f, grad_f, hess_f, x0, y0, lower, upper, keep_path = TRUE)

  data.frame(
    x0 = x0, y0 = y0,
    end_x = r$par[1],
    end_y = r$par[2],
    end_f = r$value,
    converged = isTRUE(r$converged),
    iter = r$iter
  )
}))

endpts <- unique(round(endpts[, c("end_x", "end_y", "end_f", "converged")], 6))

filled.contour(x, y, z,
  nlevels = 30,
  color.palette = hcl.colors,
  plot.title = title(main = "Attractors other than the global minimum",
    xlab = "x", ylab = "y"),
  plot.axes = {
    axis(1); axis(2)
    contour(x, y, z, add = TRUE, drawlabels = TRUE)
  }

```

```

## Endpoints of Newton from the bad starts
cols <- ifelse(endpts$converged, "red", "orange")
points(endpts$end_x, endpts$end_y, pch = 19, col = cols, cex = 1.1)

## Global minimum (reference)
points(x_star_ref[1], x_star_ref[2], pch = 8, col = "white", cex = 1.6, lwd = 2)

legend("topright",
      legend = c("Newton endpoints (converged)",
                  "Newton endpoints (not converged)",
                  "Global minimum (reference)"),
      pch = c(19, 19, 8),
      col = c("red", "orange", "white"),
      pt.cex = c(1.1, 1.1, 1.3),
      text.col = "white",
      bty = "n")
}
)
#1.4
# Since none of the detected endpoints lies near the boundary, we can assume it is not
# due to the boundary interfering with the workings of our algorithm
# We can therefore classify the endpoints by eigenvalues of H(x,y)
# - all eigenvalues >0 -> local minimum
# - mixed signs -> saddle
# - all <0 -> local maximum
classify_interior <- function(x, y, grad_f, hess_f, tol_g = 1e-6) {
  g <- grad_f(x, y)
  H <- hess_f(x, y)
  eg <- eigen(H, symmetric = TRUE)$values

  # if (sqrt(sum(g^2)) > tol_g) return("Not stationary (interior)")

  if (all(eg > 0)) "Local min"
  else if (all(eg < 0)) "Local max"
  else "Saddle / indefinite"
}

endpts$class <- apply(endpts[, c("end_x", "end_y")], 1, \(v) {
  classify_interior(v[1], v[2], grad_f, hess_f)
})

global_row <- data.frame(
  end_x = x_star_ref[1],
  end_y = x_star_ref[2],
  end_f = f(x_star_ref[1], x_star_ref[2]),
  class = "Global minimum",
  converged = TRUE
)

endpts2 <- rbind(endpts, global_row)

table(endpts2$class)

```

```
endpts2[order(endpts2$end_f), ]
```